

Documentation Technique & Fonctionnelle - Projet RickLab™

Sommaire

1. Présentation du Projet
2. Processus de Conception
3. Documentation Technique
 - a. Architecture du Projet
 - b. Composants Utilisés/ Schémas et Branchements
 - c. Structure Logicielle-archi
 - d. IA Utilisée
 - e. Explication du Code
4. Fabrication
5. Difficultés et Limites
6. Gestion de Projet

1. PRÉSENTATION DU PROJET

1.1. Problématique résolue

Description du problème interdimensionnel résolu et besoin auquel répond l'invention.

Notre idée principale était de créer un dispositif équipé de détecteurs de mouvement, qui attendrait que quelqu'un passe devant le portail pour l'activer et un buzzer signale le lancement du scan,. Le dispositif mesurerait ensuite la température ambiante et, en fonction des résultats, changerait la couleur du portail, indiquant ainsi que la destination finale avait changé.

1.2. Pertinence dans le RickLab

Justification de l'invention au sein du laboratoire de Rick Sanchez.

Notre invention s'intègre parfaitement au RickLab, car l'un des principaux défis de ce laboratoire réside dans l'apparition de portails instables et imprévisibles.

L'idée à l'origine de notre prototype est de créer un système capable de détecter la présence d'une personne, d'analyser l'environnement à l'aide de capteurs, puis de déterminer vers quelle dimension le portail doit s'ouvrir.

Cela s'inscrit parfaitement dans l'univers du laboratoire de Rick, où les technologies sont souvent expérimentales et peu conventionnelles.

2. PROCESSUS DE CONCEPTION

2.1. Recherches initiales et état de l'art

Exploration des technologies et concepts.

La distinction entre ce que nous voulions faire et ce que nous étions capables de faire. Nous avons donc eu quelques idées absurdes, mais trop irréalistes, comme l'idée de faire un objet qui change quelque chose par son parallèle, que nous avons dû abandonner. Cependant, nous devons également garder à l'esprit un concept qui s'inscrivait dans l'univers de Rick et Morty, et c'est ainsi que nous avons finalement opté pour le portail.

Nous nous sommes ensuite intéressés aux capteurs, notamment infrarouges, magnétiques et de luminosité, car nous voulions justifier le changement de localisation de notre portail. Par la suite, la LED RGB est devenue le cœur de notre projet, tout le reste s'articulant autour d'elle.

Le dernier élément sur lequel nous nous sommes concentrés était l'ESP8266, qui sert de carte principale pour la lecture des capteurs et le câblage des LED.

Nous avons ensuite dû réfléchir à la manière dont nous allions relier tous ces éléments, tant sur le plan physique qu'au niveau du code;

Détection d'une personne -> capteur IR -> buzzer -> capte la lumière -> calcule le résultat -> lisse et change la couleur du portail.

2.2. Idées abandonnées et pistes écartées

Pistes non retenues et justifications.

Au départ, nous avons prévu d'intégrer d'autres capteurs, comme un capteur sonore qui aurait eu la même fonction que le capteur de luminosité, il aurait capté les signaux sonores dans la pièce et aurait ensuite influencé la destination finale.

L'autre capteur que nous souhaitions intégrer était un capteur magnétique qui nous aurait permis d'évaluer le champ magnétique autour, voire de détecter si l'utilisateur portait un objet magnétique sur lui, mais nous avons finalement décidé de ne pas l'ajouter.

Enfin, le dernier élément que nous avons écarté concerne le design du portail : nous avons rencontré des problèmes liés à la taille et aux dimensions de l'appareil.

2.3. Évolutions du projet et choix techniques

Choix du transistor de commutation

Premier essai : transistors bipolaires TIP122 (Darlington NPN) mais ce qui était disponible. Ça n'a pas marché, les GPIO de l'ESP8266 sortent en 3V et peu de courant, pas assez pour fournir le courant de base nécessaire à la commutation du TIP122. Résultat : transistor bloqué, bandeaux éteints, quel que soit le signal envoyé.

On est passés à des MOSFET IRLB8721PBF, des logic-level faits pour être pilotés directement en 3,3-5V. Le seuil de grille ($V_{gs(th)}$) est bas, donc l'activation est franche même

avec un GPIO faible courant. La résistance à l'état passant ($R_{ds(on)}$) est très faible, donc peu de pertes et peu d'échauffement. Et la commutation est assez rapide pour un signal PWM propre ce qui compte pour éviter le scintillement en gradation.

Application au bandeau LED

Le bandeau RGB a 4 broches : alimentation commune +5V, plus trois cathodes Rouge/Vert/Bleu. Chaque canal a son MOSFET dédié (Q1, Q2, Q3), monté côté masse (low-side switching) : drain sur la cathode du canal, source à la masse commune, grille reliée au GPIO correspondant.

Choix et placement des résistances

Deux résistances par MOSFET. Une résistance de grille de $220\ \Omega$ (R2, R3, R4), en série entre le GPIO et la grille, qui limite le courant d'appel à la charge de la capacité de grille et amortit les oscillations. Une résistance de pull-down de $10\ k\Omega \pm 5\%$ (R5, R6, R7), entre la grille et la masse elle garde le MOSFET bloqué par défaut quand le GPIO est en haute impédance (démarrage, reset de l'ESP8266), pour éviter un allumage intempestif.

Les deux résistances sont placées au plus près de la grille, pour limiter la longueur de piste/fil susceptible de capter du bruit électromagnétique et dégrader le signal de commande.

Une quatrième résistance de $10\ k\Omega$ est utilisée en pont diviseur de tension avec la photorésistance (LDR) : montée entre le +3,3V et la masse, avec le point milieu relié à l'entrée analogique A0 de l'ESP8266. Ce montage convertit la variation de résistance de la LDR (dépendante de la luminosité ambiante) en une tension exploitable par le convertisseur analogique-numérique.

3. DOCUMENTATION TECHNIQUE

3.1. Architecture du projet

Cette section présente l'architecture générale du projet ainsi que les interactions entre les différents composants matériels et logiciels.

Fusion 360 :

Utilisé pour la conception et la modélisation 3D des différents éléments mécaniques du projet. Les modèles réalisés permettent notamment de définir l'intégration et l'assemblage des composants électroniques dans le boîtier.

Arduino :

Le microcontrôleur Arduino assure la gestion des différents composants électroniques. Il

recupère les informations provenant des capteurs, traite ces données et commande les différents éléments de sortie, notamment les LED et le buzzer.

EasyEDA :

Utilisé pour concevoir et documenter le circuit électronique. Le schéma permet de représenter les connexions entre le microcontrôleur, les capteurs, les LED, le buzzer et les différentes alimentations.

GitHub :

Utilisé pour le versionnement et la sauvegarde du code source ainsi que des fichiers du projet. Il permet également de suivre les modifications effectuées et de conserver un historique des différentes versions.

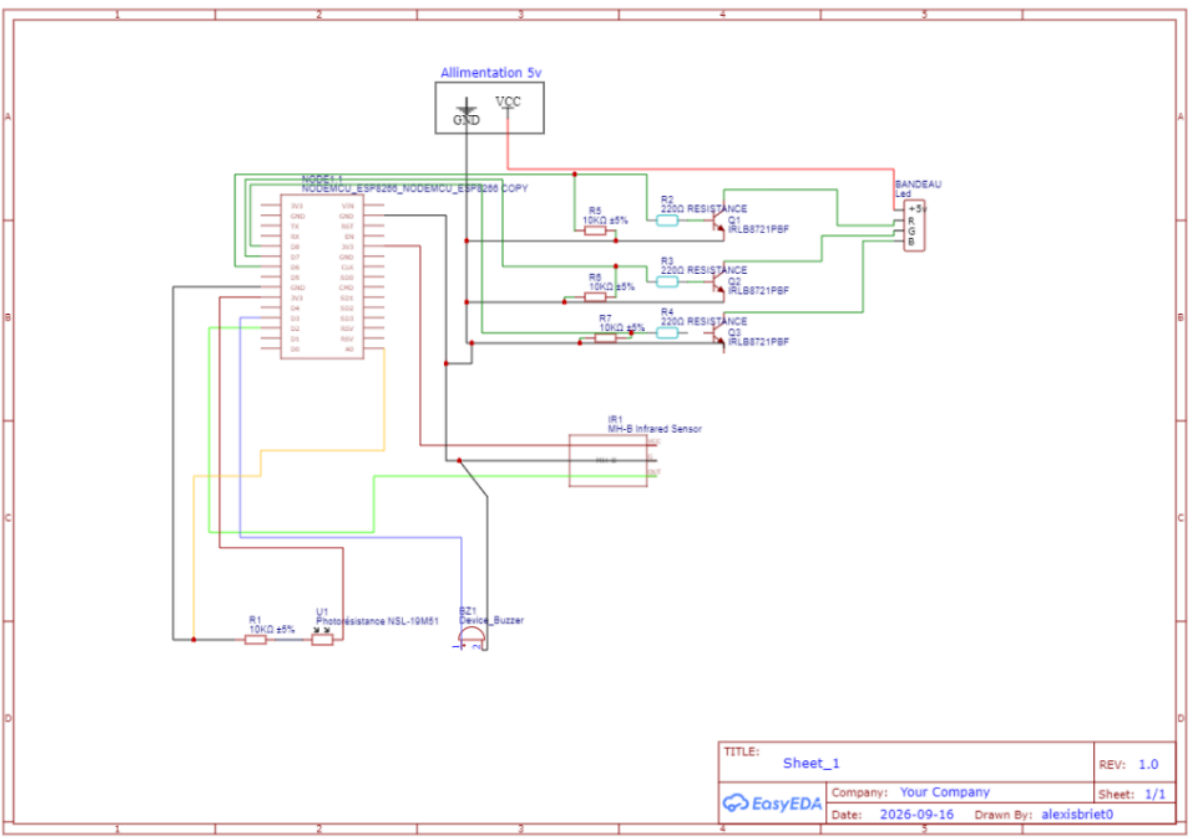
3.2. Composants Utilisés

Cette section présente les différents composants matériels utilisés pour réaliser le prototype. Les composants sont répartis entre le microcontrôleur, les capteurs, les actionneurs, les composants électroniques et les éléments mécaniques.

Composant	Quantité	Rôle/Fonction
ESP8266	1	Microcontrôleur principal. Il assure le traitement des informations provenant des capteurs et commande les différents actionneurs.
Photorésistance (LDR)	1	Mesure l'intensité lumineuse afin de détecter les variations de luminosité.
Capteur infrarouge MH-B	1	Permet de détecter la présence ou le passage d'un objet/personne grâce à un signal infrarouge.
Buzzer passif	1	Génère un signal sonore commandé par l'ESP8266.
Bande LED	1	Permet de produire un signal lumineux et d'indiquer différents états du système.
MOSFET IRLB8721	3	Permet de commander les charges nécessitant davantage de courant que ce que peut fournir directement l'ESP8266, notamment les LED.
Résistance 220 Ω	3	Limite le courant dans les circuits des LED afin de les protéger.
Résistance 10 k Ω	4	Utilisée pour le conditionnement des signaux et le maintien de niveaux logiques sur certaines entrées/sorties.
Breadboard / planche de test	1	Permet de réaliser et tester le circuit électronique sans soudure définitive.
Alimentation	1	Fournit l'énergie électrique nécessaire au fonctionnement du système.

Composant	Quantité	Rôle/Fonction
Câbles / fils de connexion	Plusieurs	Assure les connexions électriques entre les différents composants.
Plexiglas (plexi)	1	

3.3. Schémas et Branchements



Le tableau suivant détaille le câblage entre l'ESP8266 et chaque composant :

Composant	Broche ESP8266	Câblage
Capteur IR MH-B	D2 (entrée numérique)	Sortie du capteur reliée à D2 ; sortie active à l'état bas (0 = détection)
Photorésistance (LDR)	A0 (entrée analogique)	Montée en pont diviseur de tension avec une résistance fixe
Buzzer passif	D3 (sortie, piloté via tone())	Une broche sur D3, l'autre sur la masse

MOSFET canal Rouge (Q1)	D8 → grille	Résistance série 220 Ω (R2) entre D8 et la grille ; résistance de pull-down 10 k Ω entre grille et masse ; drain sur la cathode rouge du bandeau, source à la masse commune
MOSFET canal Vert (Q2)	D7 → grille	Même montage, R3
MOSFET canal Bleu (Q3)	D6 → grille	Même montage, R4
Bandeau LED RGB	—	Anode commune +5V (alimentation externe, indépendante de l'ESP8266) ; cathodes R/V/B pilotées par Q1/Q2/Q3

Chaque canal de couleur est commandé en low-side switching : le MOSFET commute côté masse plutôt que côté alimentation, ce qui permet à un GPIO 3,3V de contrôler une charge alimentée en 5V sans interface supplémentaire. La résistance de grille limite l'appel de courant au démarrage, et la résistance de pull-down garantit que le MOSFET reste bloqué par défaut (LED éteintes) au démarrage de l'ESP8266, avant que le programme n'ait initialisé les sorties.

3.4. Structure Logicielle

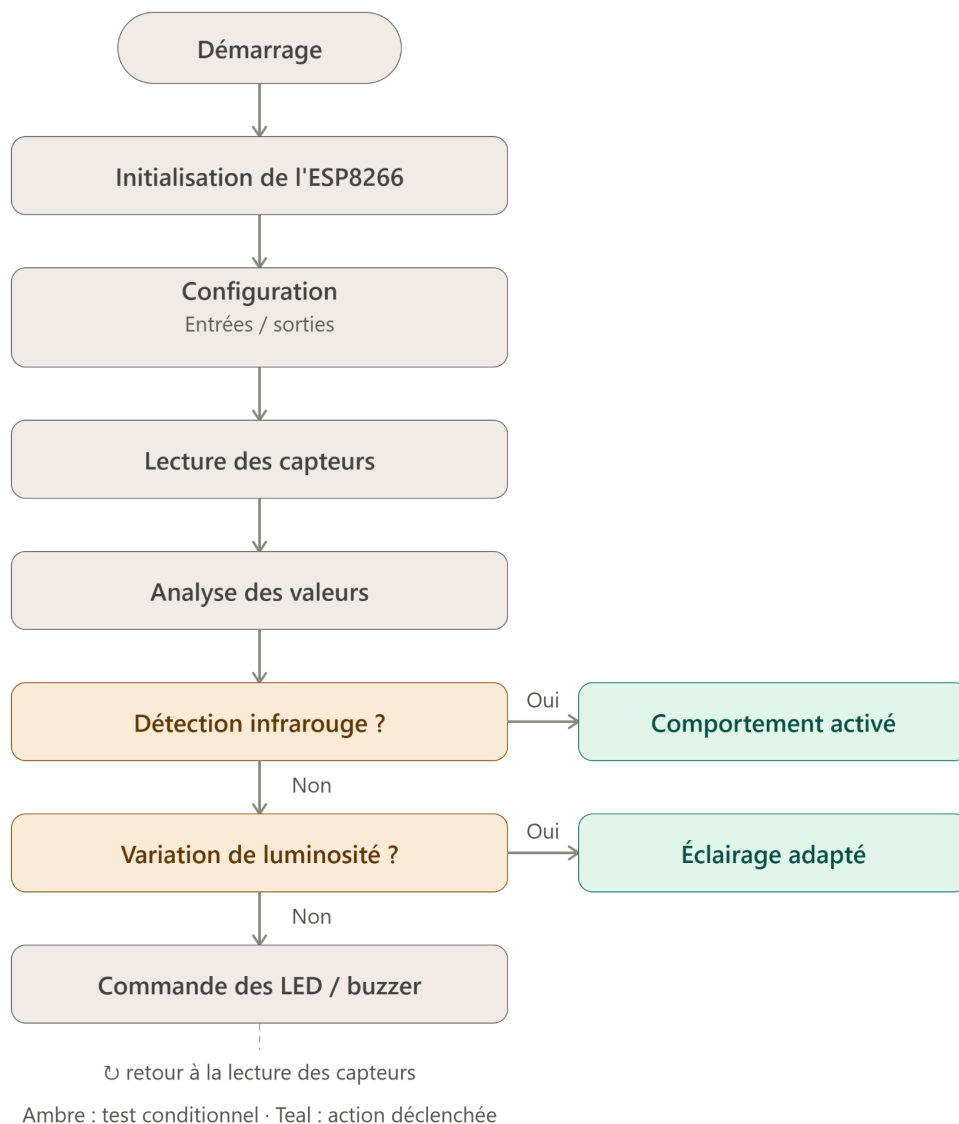
Le programme est développé en Arduino/C++ et exécuté sur l'ESP8266. Il est organisé autour de deux phases principales : l'initialisation du système et la boucle principale d'exécution.

C/C++

Projet/

```
|
|— ir_buzzer_ldr_bandeau.ino # Programme principal
|— configuration des broches # Définition des entrées/sorties
|— initialisation           # Configuration du matériel
|— lecture des capteurs     # LDR et capteur infrarouge
|— déclenchement du buzzer # signal sonore si présence détectée
par l'IR
|— calcul de la couleur # conversion de la valeur LDR en teinte
RGB
|— pilotage du bandeau LED # écriture PWM sur les 3 GPIO (via
les MOSFET)
```

Cycle d'exécution



Au démarrage, l'ESP8266 initialise les différentes entrées et sorties utilisées par le système. Il configure notamment les capteurs, les LED, les MOSFET et le buzzer.

Une fois l'initialisation terminée, le programme entre dans une boucle d'exécution continue :

Retour à la lecture des capteurs

La fonction `loop()` est exécutée en continu afin de permettre au système de réagir aux changements détectés par les capteurs en temps réel.

3.7. IA Utilisée

Nous avons utilisé ChatGPT pour la création du poster et la structuration de certaines idées, et Claude pour la génération du code.

3.8. Explication du Code

3.8.1 Logique générale

Le programme fait cohabiter deux comportements sur le même bandeau RGB :

- un mode ambiance, actif en permanence, qui fait varier la teinte du portail en fonction de la lumière ambiante mesurée par la photorésistance ;
- un mode détection, déclenché par le capteur infrarouge, qui interrompt temporairement l'ambiance pour jouer une séquence complète d'ouverture du portail (son, fondu vers le vert, effet de respiration, fondu vers le noir), puis rend la main à l'ambiance.

3.8.2 Configuration matérielle et constantes

```
const uint8_t PIN_INFRAROUGE = D2;  
const uint8_t PIN_BUZZER     = D3;  
const uint8_t PIN_PHOTORESISTANCE = A0;  
const uint8_t PIN_R = D8;  
const uint8_t PIN_G = D7;  
const uint8_t PIN_B = D6;
```

Le capteur IR est actif à l'état bas (INFRAROUGE_ACTIF = LOW). Deux constantes de temporisation stabilisent la détection : ANTI_REBOND_MS (200 ms) filtre les micro-oscillations du signal IR, et MAINTIEN_MS (1500 ms) fait qu'une personne reste considérée comme "présente" jusqu'à 1,5 s après la dernière détection stable, même si le faisceau est momentanément coupé.

PHOTORESISTANCE_MIN (57) et PHOTORESISTANCE_MAX (400) sont les bornes réelles mesurées avec notre LDR (obscurité / pleine lumière), utilisées pour calibrer la conversion en teinte.

3.8.3 Correction gamma

```
void construireTableGamma() {  
    for (int i = 0; i < 256; i++) {  
        tableGamma8[i] = (uint8_t)(pow((float)i / 255.0, EXPOSANT_GAMMA) * 255.0 + 0.5);  
    }  
}
```

L'œil perçoit la luminosité de façon non linéaire : une rampe PWM linéaire (0 à 255) paraît trop sombre sur toute sa première moitié et sature visuellement dans la seconde. La table tableGamma8 pré-calcule une correction (exposant 0,35) appliquée à chaque écriture PWM

dans `definirCouleur()`, pour un dégradé visuellement homogène. La table est construite une seule fois dans `setup()` pour éviter de recalculer `pow()` à chaque boucle — coûteux sur l'ESP8266.

3.8.4 Mode ambiance : de la LDR à la couleur

```
void effetLumiere() {  
    int brut = analogRead(PIN_PHOTORESISTANCE);  
    lueLissee += (brut - lueLissee) * LISSAGE;  
    int valeur = constrain((int)lueLissee, PHOTORESISTANCE_MIN,  
PHOTORESISTANCE_MAX);  
    float t = (float)(valeur - PHOTORESISTANCE_MIN) / (PHOTORESISTANCE_MAX -  
PHOTORESISTANCE_MIN);  
    uint16_t teinte = (uint16_t)(280 - 280 * t);  
    ...  
}
```

La lecture brute de la LDR est lissée par un filtre exponentiel (`LISSAGE = 0.05`) pour éliminer le bruit de mesure sans réagir trop lentement aux vrais changements de lumière. La valeur lissée est ramenée entre 0 et 1 (`t`), puis convertie en teinte sur la roue chromatique (`teinte = 280 - 280*t`) : plus il fait sombre, plus la teinte tend vers le violet/bleu ; plus il fait clair, plus elle tend vers le rouge, en passant par le cyan, le vert et le jaune entre les deux. `teinteVersRGB()` convertit ensuite cette teinte en triplet RGB (découpage classique de la roue en 6 secteurs de 60°), avant application via `definirCouleur()`.

3.8.5 Détection de présence et anti-rebond

```
bool lireIRAntiRebond() { ... }  
void gererCapteurIR() {  
    bool detecte = lireIRAntiRebond();  
    if (detecte && armeIR) {  
        armeIR = false;  
        animationPortail();  
    }  
    if (!detecte) armeIR = true;  
}
```

`lireIRAntiRebond()` ne valide un changement d'état qu'après 200 ms de stabilité (anti-rebond), puis maintient l'état "détecté" pendant `MAINTIEN_MS`. Le drapeau `armeIR` rend le déclenchement franc (sur front) plutôt que continu : une fois l'animation lancée, elle ne redémarre pas tant que la personne reste dans la fenêtre de maintien — il faut qu'elle s'éloigne (et que le maintien expire) avant qu'une nouvelle détection puisse relancer la séquence.

3.8.6 Séquence d'animation du portail

```
void animationPortail() {  
    sonPortail();  
    ouvrirPortail();  
}
```

```

unsigned long debut = millis();
while (millis() - debut < 2000) { portailStable(); delay(15); }
fermerPortail();
}

```

Quatre étapes successives, toutes bloquantes (~5 s au total) :

1. sonPortail() — coupe la LED, joue 4 bips ascendants (600 à 1200 Hz) via tone(), puis restaure la couleur d'ambiance précédente.
2. ouvrirPortail() — fondu progressif vers le vert du portail (RVB 20/255/90) sur ~1,2 s, avec un scintillement aléatoire par canal pour un effet instable.
3. Maintien 2 s — portailStable() fait "respirer" la couleur (oscillation sinusoïdale douce + léger scintillement) tant que le portail reste ouvert.
4. fermerPortail() — fondu inverse vers le noir sur 0,8 s.

Pendant cette séquence, le mode ambiance est mis en pause (la boucle est bloquée par les delay() internes) : c'est voulu, l'animation du portail prend la priorité sur l'éclairage d'ambiance.

3.8.7 Boucle principale

```

void loop() {
  gererCapteurIR();
  effetLumiere();
  delay(15);
}

```

À chaque itération (~15 ms, soit ~66 Hz), le programme vérifie d'abord une éventuelle détection, puis met à jour l'ambiance lumineuse si aucune animation n'est en cours. Le pas de 15 ms est cohérent avec celui utilisé dans les fondus, pour une fluidité homogène.

4. FABRICATION

4.1. Utilisation des machines du myDiL

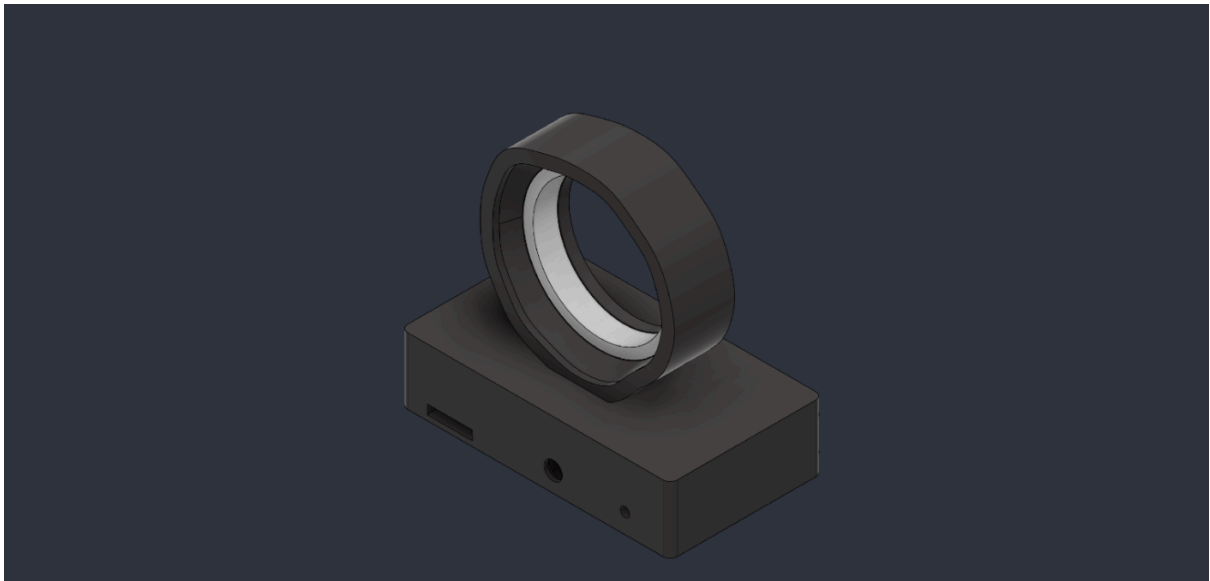
Pour la fabrication du prototype, l'équipe a principalement utilisé l'imprimante 3D la Crealty K2 du myDiL. Elle a permis de réaliser les trois pièces mécaniques du portail : la base accueillant l'ESP8266 et le câblage, la pièce circulaire formant le cadre du portail, et le cercle intérieur diffusant la lumière RGB.

On a fait ce choix car le projet nécessitait un boîtier sur mesure, à la fois pour intégrer proprement l'électronique et pour donner au prototype l'aspect visuel d'un portail.

L'impression 3D a permis d'itérer rapidement sur les dimensions, notamment après

l'abandon du premier design. Quelques imperfections de conception et d'impression ont été rencontrées (voir 5.1), ce qui a nécessité des ajustements sur les tolérances d'assemblage. Une version de la nouvelle conception a été faite parce qu'elle était trop petite, en même temps qu'elle corrigeait des soucis de modélisation des socles des composants.

4.2. Pièces imprimées ou découpées



4.3. Prototypage et assemblage

Montage mécanique et physique.



Nous avons divisé l'impression en trois parties : la première constitue la base dans laquelle nous placerons l'ESP et les câbles, mais qui servira également de support aux autres pièces imprimées ; la deuxième est la pièce circulaire qui fera office de portail ; et enfin, le cercle situé à l'intérieur, dont le rôle est de diffuser les lumières RVB.

5. DIFFICULTÉS ET LIMITES

5.1. Problèmes rencontrés

Analyse des dysfonctionnements survenus et retours d'expérience associés :

- Incompatibilité du transistor MOSFET : Le composant initialement retenu s'est avéré inadapté aux tensions et courants de commande fournis par le microcontrôleur.
- module ESP cramés : Destruction de la carte ESP en cours de manipulation ou de test (« ESP burnt »).
- Bandeau LED pas forcément fait pour ça : Problèmes d'intégration et de fonctionnement avec le ruban LED utilisé.
- Imperfections de conception et d'impression 3D : Défauts constatés lors de l'impression, des problèmes de conception et de design 3D

5.2. Améliorations Envisagées

5.2.1. Code

Le code fonctionne pour la démonstration de base, mais certains points mériteraient d'être retravaillés avant d'être considérés comme définitifs, notamment la gestion des transitions entre la lecture des capteurs (IR, LDR) et le changement de couleur du portail, pour éviter des réactions incohérentes en cas de lecture bruitée. Une relecture du code avec des tests plus systématiques (valeurs limites des capteurs, comportement au démarrage) permettrait de fiabiliser le comportement global avant une nouvelle itération.

5.2.2. Câblage (Hardware)

Le câblage a posé des problèmes concrets pendant le projet, notamment la destruction d'un ESP8266 lors des tests (cf. section 5.1). Cela vient en partie du premier choix de transistor (TIP122), incompatible avec le niveau logique 3,3V de l'ESP8266.

5.2.3. Modélisation

Le modèle 3D réalisé sous Fusion 360 n'a pas suffisamment anticipé les contraintes d'assemblage réelles : comme mentionné en 2.2, une première version du boîtier qui ne nous plaisait pas et posait des problèmes de dimensions. Le décalage vient surtout d'un

manque d'itération entre la conception et l'impression (peu de prototypes intermédiaires avant la version finale).

Pistes d'amélioration :

- Prévoir des impressions de test à chaque étape clé plutôt qu'une impression finale directe
- Intégrer les tolérances d'impression 3D dès la modélisation (jeu entre pièces, épaisseur de paroi)
- Valider l'emplacement des composants électroniques dans le boîtier avant impression définitive

5.2.4. Gestion des LED

Problème identifié : le bandeau RGB actuel ne peut pas être piloté LED par LED seul un contrôle global des trois canaux (Rouge/Vert/Bleu) est possible.

Cause réelle : ce n'est pas une limite de puissance de l'ESP8266, mais une limite du bandeau lui-même. Un bandeau RGB classique a une seule ligne de commande par couleur, commune à toutes les LED de la bande — il est donc électriquement impossible de contrôler une LED indépendamment des autres, quelle que soit la puissance disponible.

Solution recommandée :

- Remplacer le bandeau par des LED adressables (type WS2812B/NeoPixel), qui intègrent une puce de pilotage dans chaque LED et se contrôlent via un protocole série à un seul fil de données (pas du PWM ni de l'I2C)
- Dimensionner une alimentation dédiée en fonction du nombre de LED adressables utilisées, ce type de LED pouvant consommer davantage à pleine luminosité

5.3. Perspectives d'évolution

- Passer à une bande LED adressable pour un contrôle individuel des LED
- Améliorer le boîtier (tolérances d'impression, meilleure intégration des composants)
- Simplifier l'alimentation : utiliser une seule alimentation avec des abaisseurs de tension pour desservir l'ensemble du système, au lieu de plusieurs sources séparées

6. GESTION DE PROJET

6.1. Organisation de l'équipe

Structure et répartition des responsabilités au sein de l'équipe :

- Développement logiciel : Programmation de l'ESP8266 et gestion des algorithmes.
- IoT & Électronique : Intégration des capteurs, câblage et gestion des composants matériels.
- Conception 3D & Design : Modélisation et impression des structures physiques du projet.

6.2. Planification et suivi

Jour	Objectifs visés	Avancement	Problèmes techniques / Contraintes
Lundi	Phase de réflexion initiale, définition du concept de portail et distribution des rôles au sein de l'équipe.	Découvert du projet et choix du projet	La possibilité de faire ce qui est possible et ce qui est pas possible de faire
Mardi	Montage sur plaque d'essai, premières expérimentations avec les modules d'acquisition et création du dépôt de code.	Commencement des choix des composants et début des testes	Incompatibilité du transistor TIP122 (commutation impossible en 3.3V)
Mercredi	Interfaçage des capteurs avec le bandeau lumineux et élaboration de l'architecture logicielle.	On a pu continuer le projet, le câblage, mais on a commencé à faire les documents et autres.	Nécessité d'approvisionner des transistors MOSFET adaptés
Jeudi	Finalisation de l'intégration, modélisation de la structure physique et rédaction de la documentation.	On a terminé la boîte, le câblage, les documents et la vidéo.	Luminosité faible, inconvenient ESP8266
Vendredi	Préparation de la présentation orale et livrable définitive.	...	...

6.3. Outils de gestion

Pour la gestion de projet, l'équipe s'est appuyée sur **Trello**, avec un board organisé en colonnes (À faire / En cours / En test / Terminé), permettant de suivre l'avancement des tâches au fil des Daily Scrum.